

Programming Assignment 4

Course Instructors: Erkut Erdem, Hacer Yalım Keleş, Adnan Özsoy

TAs: Açelya Dalgıç, Merve Işıl Peten, Mükerrerem Aslı Taşgetiren*, Sümeyye Meryem Taşyürek*

Topics: Shortest Path, Graph-based Scheduling, Regular Expressions

Programming Language: Java (OpenJDK 11)

Due Date: **Sunday, 17.05.2026 (23:59:59)**

Spaceport Launch Preparation and Transit Network

The AstraNova Spaceport is preparing for a major orbital launch, where every operation must be executed with precision and timing. Launch day involves many interdependent preparation steps, from activating ground systems and running telemetry diagnostics to fueling modules and granting final launch authorization. At the same time, engineers and technicians must move efficiently across the spaceport between hangars, control units, launch pads, and service terminals.

You, as a talented computer engineering student, have been recruited to support both the launch preparation process and the internal transit system of the spaceport.

Thanks to your reputation for clean algorithms and efficient code, you've been given two important missions:

1. **Launch Preparation Planner:** Help coordinate all launch-related preparation steps by creating a schedule that respects prerequisite constraints and ensures the earliest possible mission readiness.
2. **Spaceport Transit Network:** Help personnel travel efficiently across the spaceport by finding the fastest route between two locations using the best combination of walking and autonomous shuttle lines.



These missions rely on your understanding of graph algorithms and regular expressions, and your ability to implement them in Java using a provided starter codebase. Your work will help ensure that launch operations are completed on time and that movement across the spaceport remains fast, reliable, and efficient.

1 Part I – Launch Operations Timeline

Before a launch window opens, the spaceport must complete a set of operational procedures in a valid order. Some procedures can be initiated immediately, while others remain blocked until specific earlier procedures have been completed. Since even a small delay may affect the full launch sequence, mission control requires a software component that can compute a valid earliest-start timeline for all required operations.

In this part of the assignment, you will implement a planner that analyzes prerequisite-constrained launch operations and produces the earliest feasible execution timeline for each

mission. These operations represent the final phase of launch preparation, where tasks are executed in a tightly scheduled time window prior to launch.

1.1 Operational Context and Objective

Consider the preparation workflow of the *Orbiter Alpha* mission. The launch plan includes multiple operations such as powering ground support units, validating telemetry, fueling the core stage, attaching the payload, and issuing the final authorization signal. For each operation, the input specifies:

- an **operation code**,
- an **operation label**,
- an **execution time** (in hours),
- and the list of operations that **must be completed beforehand**.

A sample launch plan is shown below:

Operation Code	Operation Label	Execution Time	Blocked By
0	Activate ground support units	2 h	-
1	Validate telemetry link	1 h	0
2	Fuel core stage	3 h	0
3	Attach payload module	2 h	2
4	Run integrated systems check	1 h	1, 3
5	Issue launch authorization	1 h	4

Table 1: Orbiter Alpha launch operations

For example, Operation 4 (*Run integrated systems check*) cannot begin until both Operation 1 (*Validate telemetry link*) and Operation 3 (*Attach payload module*) have finished. Operation 0 can be started immediately, while Operation 5 is the final step before the mission becomes ready for launch.

Your goal is to compute the **earliest activation time** and **earliest completion time** of every operation, and to determine the minimum total time required for the launch plan to reach readiness.



Objective: For each launch plan, generate an earliest-feasible execution timeline that satisfies all prerequisite constraints and minimizes total launch-readiness time.

1.2 Input File Format

The input for this part is given as an `.xml` file, passed as the first command-line argument to your program.

A single file may contain multiple launch plans. Its structure is as follows:

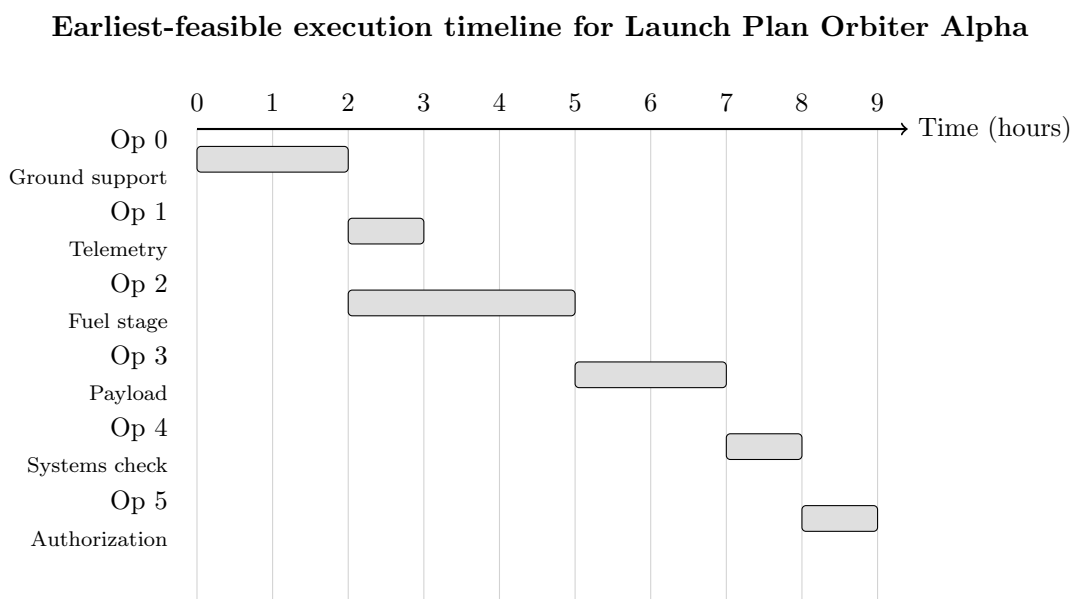
```
<LaunchPlans>
```

```
<LaunchPlan>
  <PlanName>Orbiter Alpha</PlanName>
  <Operations>
    <Operation>
      <Code>0</Code>
      <Label>Activate ground support units</Label>
      <ExecutionTime>2</ExecutionTime>
      <BlockedBy></BlockedBy>
    </Operation>
    <Operation>
      <Code>1</Code>
      <Label>Validate telemetry link</Label>
      <ExecutionTime>1</ExecutionTime>
      <BlockedBy>
        <RequiresOperation>0</RequiresOperation>
      </BlockedBy>
    </Operation>
    ...
  </Operations>
</LaunchPlan>
</LaunchPlans>
```

Each launch plan contains a list of operations. Each operation has a unique integer code, a textual label, an execution time in hours, and zero or more prerequisite operations.

1.3 Expected Solution and Output Format

For the launch plan **Orbiter Alpha**, one valid earliest-feasible timeline is illustrated below:



In this timeline, Operations 0, 2, 3, 4, and 5 form the *critical chain*: delaying any of them would delay the full launch-readiness time. Operation 1 is not on this chain and has limited flexibility.

Your program must compute and report the earliest begin and finish times of all operations. For the example above, the result is:

```
+-----+
Operation  | 0 | 1 | 2 | 3 | 4 | 5 |
+-----+
Begin time  | 0 | 2 | 2 | 5 | 7 | 8 |
+-----+
Finish time | 2 | 3 | 5 | 7 | 8 | 9 |
+-----+
Launch readiness time: 9 hours
```

The final report must include the launch plan name, the earliest begin and finish times of each operation, and the total time required for the mission to become ready. Operations must appear in a valid prerequisite-respecting order.



You are expected to complete the following methods:

- `readXML()` and `printTimeline()` in the `LaunchOperationsTimeline` class
- `getEarliestTimeline()` and `getReadinessTime()` in the `LaunchPlan` class

The expected STDOUT output format for a single launch plan is given below:

```
=== LAUNCH OPERATIONS REPORT START ===
-----
Launch Plan: Orbiter Alpha
-----
Code      Operation                                Begin   Finish
-----
0         Activate ground support units              0       2
1         Validate telemetry link                   2       3
2         Fuel core stage                           2       5
3         Attach payload module                    5       7
4         Run integrated systems check             7       8
5         Issue launch authorization              8       9
-----
Launch-ready in 9 hour(s).
-----
=== LAUNCH OPERATIONS REPORT END ===
```

The output must match this format exactly. Any deviation in ordering, spacing, or report structure may result in point loss during automated grading.

2 Part II – Spaceport Transit Planner

Once the launch timeline has been prepared, mission control must also coordinate movement across the spaceport. The AstraNova facility includes launch pads, service bays, engineering terminals, control centers, and maintenance zones spread over a wide area. To move between these locations efficiently, personnel may either walk directly or use autonomous shuttle corridors that connect selected stations.

In this part of the assignment, you will implement a **Spaceport Transit Planner** that computes the *fastest route* between two points inside the facility.

2.1 Transit Model and Objective

Personnel moving across the spaceport have two transportation options:

- **Walking:** movement is allowed directly between any two points on the map.
- **Autonomous shuttle corridors:** shuttles travel along predefined directed corridors connecting listed stations.

Walking is always available, but shuttle travel is usually faster and may significantly reduce the total transit time when used effectively. Your task is to combine these transportation modes and determine the minimum-time route between a given origin and destination.



Objective: Given the coordinates of an origin point and a destination point, together with the shuttle corridor definitions, compute the fastest route across the spaceport and print the corresponding route instructions.

Assumptions:

- Shuttle corridors operate only in the listed direction, between consecutive stations.
- Walking is allowed between any two points; average walking speed is **10 km/h**.
- Shuttle speed (in km/h) is provided in the input file.
- Waiting time is ignored.

2.2 Input File Format and Regular Expression Parsing

The input for this part will be given as a `.dat` file, passed as the second command-line argument to your program. The file contains:

- the number of shuttle corridors in the spaceport,
- the coordinates of the origin and destination points,
- the station coordinates for each shuttle corridor,
- the average shuttle speed.

A sample input file is shown below:

```
num_shuttle_corridors = 2
origin_point = (0, 0)
destination_point = (12000, 1500)
average_shuttle_speed = 500.00
corridor_name = "Orbital Link"
corridor_stations = (0, 300) (4000, 300) (7000, 300)
corridor_name = "Command Axis"
corridor_stations = (2500, 1000) (7000, 1000) (12000, 1000)
```

Stations are listed in travel order, and each corridor contains at least two stations. A shuttle may move from Station i to Station $i + 1$, but not in the reverse direction unless a separate corridor definition provides that connection. All coordinates represent points on a 2D map of the spaceport, measured in meters.

Important: The order of variables and the placement of whitespace characters (tabs or spaces) around variable names, equal signs, commas, parentheses, and quoted strings may vary. Your implementation **MUST** therefore use **regular expressions** to parse the input robustly.

You are expected to implement several parsing methods in the `SpaceportTransitNetwork` class. The following example illustrates the general idea for extracting an integer variable:

```
public int getIntVar(String varName) {
    Pattern p = Pattern.compile("[\\t ]*" + varName +
        "[\\t ]*=[\\t ]*([0-9]+)");
    Matcher m = p.matcher(fileContent);
    m.find();
    return Integer.parseInt(m.group(1));
}
```

This method uses a regular expression that tolerates flexible spacing around the assignment operator.

You should apply similar logic in the following methods:

- `getStringVar()`: Modify the regular expression so that it captures the value of the given variable as `Group 1` and returns it as a `String`.
- `getDoubleVar()`: Modify the regular expression so that it matches floating-point values in formats such as 5, 5.2, or 5.0002, and returns the result as a `Double`.
- `getPointVar()`: Modify the regular expression so that it captures two integer coordinates inside parentheses as `Group 1` and `Group 2`, then returns them as a `Point`.

2.3 Expected Solution and Output Format

Your program must print the total travel time of the fastest route to `STDOUT`, together with the step-by-step route instructions.

The total travel time must be rounded to the nearest minute. However, the durations shown

in the individual route steps **must not be rounded to integers**; they must be printed as doubles with exactly **two (2)** digits after the decimal point.

For the sample input above, the expected output is as follows:

```
=== TRANSIT PLAN START ===
Fastest route takes 10 minute(s).
Route Instructions
-----
1. Walk from "Origin Point" to "Orbital Link Station 1"
for 1.80 minutes.
2. Take the shuttle from "Orbital Link Station 1" to
"Orbital Link Station 2" for 0.48 minutes.
3. Take the shuttle from "Orbital Link Station 2" to
"Orbital Link Station 3" for 0.36 minutes.
4. Walk from "Orbital Link Station 3" to
"Command Axis Station 2" for 4.20 minutes.
5. Take the shuttle from "Command Axis Station 2" to
"Command Axis Station 3" for 0.60 minutes.
6. Walk from "Command Axis Station 3" to
"Destination Point" for 3.00 minutes.
=== TRANSIT PLAN END ===
```



You are expected to complete the following methods:

- `getStringVar()`, `getDoubleVar()`, `getPointVar()`,
`getShuttleCorridors()`, and `readInput()` in the class
`SpaceportTransitNetwork`
- `getFastestRouteInstructions()` and
`printRouteInstructions()` in the class
`SpaceportTransitPlanner`

Your solution will be tested using pre-written test files and auto-graded. Make sure that your output matches the expected format **exactly**.

Grading Policy

- Implementation of the algorithms: 90%
 - Part I – Launch Operations Timeline: 40%
 - Part II – Spaceport Transit Planner: 50%
 - * Regular Expression Parsing: 15%
 - * Fastest Route Computation: 35%
- Output format test: 10%

Run Configuration

Your code will be compiled and run as follows:

```
javac *.java  
java Main <launchPlansXMLFile> <spaceportTransitDATFile>
```

Important Notes

- Do not miss the deadline: **Sunday, 17.05.2026 (23:59:59)**.
- Save all your work until the assignment is graded.
- The assignment solution you submit must be your original, individual work. Duplicate or highly similar submissions will be considered academic dishonesty.
- You can ask your questions via Piazza (<https://piazza.com/hacettepe.edu.tr/spring2026/bbm204>), and you are responsible for following all official course announcements and discussions posted there.
- You will submit your work via <https://submit.cs.hacettepe.edu.tr/> with the file hierarchy given below:
 - b<studentID>.zip
 - * Main.java <FILE>
 - * LaunchOperationsTimeline.java <FILE>
 - * LaunchPlan.java <FILE>
 - * Operation.java <FILE>
 - * SpaceportTransitNetwork.java <FILE>
 - * SpaceportTransitPlanner.java <FILE>
 - * ShuttleCorridor.java <FILE>
 - * Station.java <FILE>
 - * Point.java <FILE>
 - * Edge.java <FILE>
 - * RouteInstruction.java <FILE>
- The name of the class containing the main method must be Main.java. You MUST use **this starter code**. The main class and all other classes must be placed directly inside the zip archive. You may create additional helper classes if needed, provided that they are also included in the same zip file.
- This file hierarchy must be submitted as a .zip archive (not .rar; only .zip files are supported).
- **Usage of any external libraries is forbidden.**

Academic Integrity Policy

All work on assignments **must be done individually**. You are encouraged to discuss the given assignments with your classmates, but these discussions should be carried out in an abstract way. That is, discussions related to a particular solution to a specific problem (either in actual code or in pseudocode) **will not be tolerated**. In short, turning in someone else's work (including work available on the internet), in whole or in part, as your own will be considered as **a violation of academic integrity**. Please note that the former condition also holds for online/AI sources. Make use of them responsibly, refrain from generating the code you are asked to implement. Remember that we also have access to such tools, making it easier to detect such cases.



The submissions will be subjected to a similarity check. Any submissions that fail the similarity check will not be graded and will be reported to the ethics committee as a case of academic integrity violation, which may result in suspension of the involved students.